

AAMT-WP-09: Compile-Gated DPO

AAMT-WP-09 | AsAManThinks Research | May 2026 | DOI:
10.5281/zenodo.19600795

Compile-Gated DPO: Executable Verification as an Automated Preference Signal for Game-Engine Code Generation

Weslyn Whitehead Jr.

AsAManThinks Research, Berkeley, CA, USA |

weslyn@asamanthinks.com

ORCID: [to be registered](#) | License: CC BY 4.0

Compile-Gated DPO: Executable Verification as an Automated Preference Signal for Game-Engine Code Generation

Authors: Weslyn Whitehead Jr.¹

Affiliations:

¹ AsAManThinks Research, Berkeley, CA, USA

Corresponding author: weslyn@asamanthinks.com

ORCID: <https://orcid.org/0009-0005-7707-3210>

Submitted: May 2026

Working Paper Series: AAMT-WP-09

Anchor DOI: 10.5281/zenodo.19600795

License: Creative Commons Attribution 4.0 International (CC BY 4.0)

Abstract

Direct Preference Optimization (DPO) requires (prompt, chosen, rejected) triplets as training signal. For general-purpose language generation, these triplets require human annotation or a reward model. For code generation targeting a specific runtime (C# in the Unity game engine), the compiler is a free, deterministic, and highly informative preference oracle: compilation success produces chosen candidates; compilation failure with structured diagnostics produces rejected candidates. We present **Compile-Gated DPO (CG-DPO)**, a system architecture for automatically generating DPO training pairs from compiler output. The current implementation — `CSharpCompileRateSuite` in MaiaM Alchemist v0.4.2 — fully realizes the compiler evaluation pipeline: writing model-generated completions to a scratch namespace, running a three-tier compile

path (full Unity dotnet → UnityEngineShim → static analysis), capturing structured error diagnostics, and persisting results to the `code_eval_compile_results` DuckDB table for run-over-run tracking. The DPO pair emission stage — converting accumulated compile results into (prompt, chosen, rejected) triplets with error-conditioned rejection context — is the next planned development step and is fully specified in Section 3. The diagnostics-as-rejection-context design, where compiler error messages are appended to the rejected completion, is the key architectural contribution: it teaches the model not just what failed but why, encoding API usage rules and type system constraints without any human annotation.

Keywords: DPO, code generation, automated evaluation, Unity, C#, compilation, preference learning, game engine AI, program synthesis

1. Introduction

The fundamental challenge of preference learning for code generation is obtaining preference signal without human annotation. Prior work has used:

- **Unit test execution (CodeRL, PPOCoder):** Tests pass/fail as reward. Requires pre-written tests.
- **Program equivalence (AlphaCode):** Compare output to reference solutions. Requires reference solutions.
- **Execution traces (CodeBERT, PLUR):** Symbolic execution for partial correctness. Complex infrastructure.
- **Human annotation:** Most reliable but expensive and slow.

For game-engine code generation specifically, the game engine’s compiler provides a richer signal than unit test execution alone:

1. The compiler detects **type errors** (wrong argument types, missing casts) that a unit test might never exercise.
2. The compiler detects **API misuse** (calling deprecated APIs, accessing internal members, missing assemblies) that is specific to the engine SDK.
3. The compiler detects **namespace errors** (missing `using` statements, ambiguous types across assemblies) that are invisible to generic linters.
4. The runtime test framework detects **logic errors** (NullReferenceException, IndexOutOfRangeException) at a fine granularity.

CG-DPO captures all four error classes as structured rejection signals and formats them as DPO triplets without any human involvement.

2. Implementation Architecture

2.1 Two-tier evaluation

The `CSharpCompileRateSuite` implements two code paths depending on tool availability:

Tier 1 — Full dotnet build (preferred):

```

_CSPROJ_TEMPLATE = """<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <OutputType>Library</OutputType>
    <TargetFramework>netstandard2.1</TargetFramework>
    <Nullable>disable</Nullable>
    <NoWarn>CS0246;CS0234;CS0103;CS0118</NoWarn>
    <GenerateAssemblyInfo>>false</GenerateAssemblyInfo>
    <EnableDefaultCompileItems>>false</EnableDefaultCompileItems>
  </PropertyGroup>
  <ItemGroup>
    {compiles}
  </ItemGroup>
  <ItemGroup>
    {references}
  </ItemGroup>
</Project>"""

```

Several template settings deserve explanation. `Nullable>disable` is intentional: the `UnityEngineShim` stub (used in the Tier 1 fallback path) does not implement full nullable annotations, and enabling nullable analysis would produce spurious warnings that mask real errors. `NoWarn` suppresses four warning codes that are expected to fire harmlessly against the shim's incomplete API surface — these same codes produce genuine errors against the full Unity managed DLLs, so the shim path suppresses them while the real Unity path resolves them via actual references. `EnableDefaultCompileItems>>false` is required because the suite writes the candidate file explicitly via `<Compile Include=...>`.

The suite writes the model's C# completion to a temporary `.cs` file with the appropriate `.csproj`, runs `dotnet build`, captures `stdout/stderr`, and parses the compiler output for error codes and messages.

Tier 2 — Static fallback with `UnityEngineShim`:

For CI environments without Unity installed, the suite uses a bundled `UnityEngineShim.cs` that stubs the Unity API surface (`MonoBehaviour`, `Transform`, `GameObject`, common methods) and runs `dotnet build` against it. This catches most type errors, namespace issues, and syntax errors without requiring a Unity installation.

Tier 3 — Pure static analysis:

If `dotnet` is not available, the suite falls back to a static analyzer that checks for class declarations, balanced braces and parentheses, `using UnityEngine` for `MonoBehaviour` classes, and the absence of stub markers (`TODO/FIXME`):

```
def _static_csharp_validate(code: str) -> tuple[bool, str, str, str]:
    if not code.strip():
        return False, "empty_response", "no code returned", "static"
    if re.search(r'\b(TODO|FIXME|stub|\.\.\.)\b', code):
        return False, "stub_response", "contains TODO/FIXME/stub", "static"
    if not re.search(r'\bclass\s+\w+', code):
        return False, "no_class", "no class declaration found", "static"
    if "MonoBehaviour" in code and "using UnityEngine" not in code:
        return False, "missing_using", "MonoBehaviour without using UnityEngine;", "static"
    # Balanced-brace and balanced-paren checks (catches truncated responses)
    ...
```

The suite records which tier was used in `result.details["mode"]` so analysis can condition on evaluation mode.

2.2 Error classification

Compiler errors are classified into five categories stored in the `error_kind` column. The actual classification uses snake_case string identifiers (not enum values) to remain serializable directly to DuckDB:

```
def _classify_error_kind(line: str) -> str:
    """Map a compiler diagnostic line to a canonical error_kind string."""
    if "CS0246" in line or "CS0234" in line or "CS0103" in line:
        return "missing_type_or_namespace" # unknown type / missing using
    if "CS1061" in line or "CS0117" in line:
        return "missing_member" # method/property does not exist on type
    if "CS1002" in line or "CS1513" in line or "CS1519" in line:
        return "syntax_error" # missing semicolon, unexpected token
    if "CS0266" in line or "CS0029" in line or "CS0019" in line:
        return "type_mismatch" # cannot convert / no operator defined
    if "CS0168" in line or "CS0219" in line:
        return "unused_variable" # declared but never used (warning-as-error)
    return "other"
```

The `error_kind` string and the first 200 characters of the compiler diagnostic line are stored in `error_kind` and `error_excerpt` respectively. Both fields are available for downstream DPO pair construction and training-time analysis.

The error kind and the full diagnostic message are preserved for DPO pair construction.

3. DPO Pair Construction

Implementation note: The `CSharpCompileRateSuite` currently accumulates compile results in the `code_eval_compile_results` DuckDB table, providing the raw signal for DPO pair construction. Automated pair emission — writing `(prompt, chosen, rejected)`

triplets to a training file — is the next planned stage of the pipeline and is not yet implemented in v0.4.2. The schema and format described in this section specify the intended output of that stage.

3.1 Chosen examples

When a completion compiles without errors (and passes runtime tests if applicable), the pair constructor emits:

```
{
  "prompt": "Write a Unity 6 MonoBehaviour that rotates a GameObj
ect at a configurable speed.",
  "chosen": "using UnityEngine;\n\npublic class Rotator : MonoBeh
aviour\n{\n    [SerializeField] float speed = 90f;\n    void Upda
te() { transform.Rotate(Vector3.up, speed * Time.deltaTime); }\n}
\n",
  "metadata": { "compiled": true, "compiler": "dotnet_unity" }
}
```

3.2 Rejected examples (diagnostic DPO)

When a completion fails to compile, the error excerpt and `error_kind` already stored in DuckDB are used to construct the rejection context:

```
{
  "prompt": "Write a Unity 6 MonoBehaviour that rotates a GameObj
ect at a configurable speed.",
  "rejected": "using UnityEngine;\n\npublic class Rotator : MonoB
ehaviour\n{\n    void Update() { transform.rotate(Vector3.Up, spe
ed * Time.deltaTime); }\n}\n\n// COMPILER ERROR [CS1061]: 'Transfo
rm' does not contain a definition for 'rotate'. Did you mean 'Rot
ate'? [missing_member]\n\n// COMPILER ERROR [CS0103]: The name 'spee
d' does not exist in the current context [missing_type_or_namespa
ce]",
  "metadata": { "compiled": false, "error_kind": "missing_member"
, "compiler": "dotnet_unity" }
}
```

The diagnostic messages are appended to the rejected completion in a structured comment block. This produces **error-conditioned rejection examples** that teach the model the specific failure mode — not just that the code was wrong but why it was wrong and where.

3.3 Why diagnostic conditioning matters

Standard DPO pairs tell the model: “prefer A over B.” Error-conditioned pairs tell the model: “prefer A over B, and B’s problem was specifically [CS1061] — wrong method name case.” The diagnostic signal allows the model to learn orthographic and API usage rules without any additional supervision — the compiler error messages encode these rules explicitly.

4. Scratch Namespace Safety

The eval suite enforces strict namespace isolation:

```

SCRATCH_NAMESPACE = "Assets/MaiiaM/Eval/Generated"
FORBIDDEN_NAMESPACES = [
    "Assets/MaiiaM/Core",
    "Assets/MaiiaM/Systems",
    "Assets/MaiiaM/Editor",
]

def write_eval_file(completion: str, filename: str) -> Path:
    assert SCRATCH_NAMESPACE in filename, "write outside scratch namespace"
    assert not any(fn in filename for fn in FORBIDDEN_NAMESPACES)
    path = eval_root / SCRATCH_NAMESPACE / filename
    path.parent.mkdir(parents=True, exist_ok=True)
    path.write_text(completion)
    return path

```

The assertion is a hard guard — any write attempt outside the scratch namespace raises immediately rather than silently writing to a protected path. The scratch namespace is added to `.gitignore` in the platform repository.

5. Results Persistence and Tracking

Compile results are persisted to the `code_eval_compile_results` DuckDB table:

```

CREATE TABLE code_eval_compile_results (
    run_id      TEXT      NOT NULL,
    suite       TEXT      NOT NULL,    -- short suite identifier,
    e.g. "csharp_compile_rate"
    sample_idx  INTEGER NOT NULL,
    language    TEXT      NOT NULL,    -- e.g. "csharp", "gls1"
    compiled    BOOLEAN NOT NULL,      -- TRUE = success, FALSE =
failure
    compiler    TEXT      NOT NULL,    -- "dotnet_unity" | "dotnet
_shim" | "static"
    duration_ms REAL,
    error_kind  TEXT,                  -- e.g. "missing_type_or_na
mespace", "missing_member"
    error_excerpt TEXT,                -- first 200 chars of compi
ler diagnostic
    api_surface TEXT,                  -- Unity API surface (e.g.
"Transform.Rotate")
    PRIMARY KEY (run_id, suite, sample_idx)
);

```

Note on column naming: `suite` (not `suite_name`), `compiled` `BOOLEAN` (not `compile_status` `TEXT`), `duration_ms` (not `elapsed_ms`), and `error_kind` (not `error_class`) reflect the actual migration in `v002_unity_and_experts.sql`. The additional columns `language`, `compiler`, `error_excerpt`, and `api_surface` carry richer diagnostic context than a single status string. The `compiler` column records which evaluation tier ran (full Unity dotnet, shim, or static), allowing downstream analysis to stratify results by evaluation fidelity.

Run-over-run comparison via the `compare.py` statistical module (Cohen's d, paired t-tests) tracks compile rate as a first-class metric alongside HeartScale coherence and archetype alignment. A promotion gate requires `compile_rate_delta` `>= -0.02` (no more than 2% regression in compile rate allowed across training runs).

6. Eval Suite Integration

The `CSharpCompileRateSuite` is one of eight Unity-specific suites in the `suites_unity` registry:

<code>CSharpCompileRateSuite</code>	← this paper
<code>ShaderCompileRateSuite</code>	← GLSL/HLSL compile via <code>glslangValidator</code>
<code>UnityApiCorrectnessSuite</code>	← static API surface correctness
<code>PrefabCompletionSuite</code>	← JSON prefab schema validation
<code>SpawnerPatternMatchSuite</code>	← GeNa/Gaia spawn config pattern matching
<code>AssetRetrievalRecallSuite</code>	← asset search recall@k
<code>OneJsJsxLintSuite</code>	← Preact/JSX lint for OneJS components
<code>VortexMathCorrectnessSuite</code>	← deterministic TERA arithmetic verification

All suites implement graceful degradation: if the required tool (`dotnet`, `glslangValidator`) is not available, they fall back to static analysis and record the mode in `result.details["mode"]`.

7. Relationship to Prior Work

CodeRL (Le et al., 2022): Uses unit test execution as the reward signal for RL fine-tuning. CG-DPO uses compilation as a primary signal (richer than test execution) and formats pairs for DPO rather than RL.

PPOCoder (Shojaee et al., 2023): PPO with execution-based reward for code generation. CG-DPO avoids RL instability by using DPO’s offline preference learning formulation.

SWE-bench (Jimenez et al., 2024): Software engineering benchmarks using test execution. CG-DPO targets game-engine-specific APIs and generates training pairs rather than benchmarks.

AlphaCode (Li et al., 2022): Large-scale code generation with competitive programming compilation/execution. CG-DPO targets a specific SDK domain (Unity C#) and includes the diagnostic message in the rejection signal.

Key distinction: CG-DPO is the first system to use game-engine compiler diagnostics as error-conditioned DPO rejection signals, to implement a three-tier fallback path (Unity dotnet → shim → static) for CI portability, and to persist compile results as a first-class tracked metric in a columnar eval store.

8. Conclusion

CG-DPO demonstrates that domain-specific compilers are powerful, free preference oracles for code generation fine-tuning. The diagnostic conditioning of rejection examples — appending structured compiler error messages to the rejected completion — provides a training signal that encodes API usage rules, namespace conventions, and type system constraints without any human annotation. The three-tier fallback architecture ensures the evaluation is portable across development environments from full Unity installations to minimal CI

containers. The result is an automated alignment loop for game-engine code generation that improves with each training run as the compile results database accumulates cross-run comparison data.

Acknowledgments

The author thanks the AsAManThinks Research and MaiiaM Platform development communities.

Funding

Self-funded through AsAManThinks Research.

Data Availability

The `CSharpCompileRateSuite` is implemented in `packages/eval-suite/eval_suite/suites_unity/csharp_compile_rate.py` in MaiiaM Alchemist v0.4.1. The `UnityEngineShim.cs` stub is bundled with the eval suite.

Conflict of Interest

The author is the founder and CEO of AsAManThinks Research.

References

- Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., & Narasimhan, K. (2024). SWE-bench: Can language models resolve real-world GitHub issues? In *Proceedings of ICLR 2024*. <https://doi.org/10.48550/arXiv.2310.06770>
- Le, H., Wang, Y., Gotmare, A. D., Savarese, S., & Hoi, S. C. H. (2022). CodeRL: Mastering code generation through pretrained models and deep reinforcement learning. In *Advances in Neural Information Processing Systems 35* (pp. 21314–21328). <https://doi.org/10.48550/arXiv.2207.01780>
- Li, Y., Choi, D., Chung, J., Kushman, N., Schrittwieser, J., Leblond, R., Eccles, T., Keeling, J., Gimeno, F., Dal Lago, A., Hubert, T., Choy, P., de Masson d’Autume, C., Babuschkin, I., Chen, X., Huang, P., Welbl, J., Goyal, S., Cherepanov, A., ... Vinyals, O. (2022). Competition-level code generation with AlphaCode. *Science*, 378(6624), 1092–1097. <https://doi.org/10.1126/science.abq1158>
- Rafailov, R., Sharma, A., Mitchell, E., Ermon, S., Manning, C. D., & Finn, C. (2023). Direct preference optimization: Your language model is secretly a reward model. In *Advances in Neural Information Processing Systems 36*. <https://doi.org/10.48550/arXiv.2305.18290>
- Shojaee, M., Jain, N., Zheng, W., Ding, T., Jain, S., Awasthi, A., & Iyer, S. (2023). Execution-based code generation using deep reinforcement learning. *arXiv preprint*. <https://doi.org/10.48550/arXiv.2301.13816>
- Whitehead, W. (2026). MaiiaM Wings: Consciousness-Aligned Language Architecture. *Zenodo*. <https://doi.org/10.5281/zenodo.19600795>